

LAPORAN PRAKTIKUM
STRUKTUR DATA
“DOUBLE LINKED LIST”



Disusun oleh:

Ndaru Pradana Gatot Suseno
2411532007

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena Laporan Praktikum Struktur Data 6 dapat diselesaikan. Laporan ini mendokumentasikan penerapan Double Linked List pada bahasa Java.

Praktikum keenam membahas node dengan dua referensi, yaitu next dan prev, sehingga data dapat ditelusuri dalam dua arah. Operasi yang diuji meliputi penelusuran maju-mundur, penambahan, dan penghapusan node. Pada laporan tugas, Double Linked List diterapkan untuk membangun playlist musik.

Penulis menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingan yang diberikan. Isi laporan disusun berdasarkan source code package pekan6_2411532007 dan hasil pengujian secara langsung. Kritik dan saran sangat diharapkan untuk meningkatkan mutu laporan.

Padang, 2026

Ndaru Pradana G.S

DAFTAR ISI

Contents

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
Latar Belakang	4
Tujuan Praktikum.....	4
Manfaat Praktikum.....	4
BAB II PEMBAHASAN	5
Dasar Teori.....	5
Program NodeDLL_2411532007 dan InsertDLL_2411532007	6
Program PenelusuranDLL_2411532007	9
Program HapusDLL_2411532007	10
BAB III KESIMPULAN.....	12
DAFTAR PUSTAKA	13
LAPORAN TUGAS	14
Tujuan Tugas.....	14
Class lagu_2411532007	15
Class musik_2411532007	16
Hasil Eksekusi Laporan Tugas.....	18
Analisis Hasil	18
Kesimpulan Laporan Tugas	18
Saran.....	19

BAB I

PENDAHULUAN

Latar Belakang

Double Linked List merupakan linked list yang setiap node-nya mempunyai pointer next dan prev. Pointer next mengarah ke node berikutnya, sedangkan prev mengarah ke node sebelumnya. Struktur ini memungkinkan traversal dari head ke tail maupun dari tail ke head.

Keberadaan dua pointer memudahkan penghapusan dan penelusuran dua arah, tetapi setiap operasi harus menjaga konsistensi kedua hubungan tersebut. Praktikum ini menguji pembaruan next dan prev pada penambahan serta penghapusan, kemudian menerapkannya pada playlist yang dapat ditampilkan maju dan mundur.

Tujuan Praktikum

1. Memahami struktur node DLL dengan next dan prev.
2. Melakukan traversal maju dan mundur.
3. Menambahkan node di awal, akhir, dan posisi tertentu.
4. Menghapus head, tail, dan node pada posisi tertentu.
5. Menerapkan DLL pada playlist musik.

Manfaat Praktikum

Praktikum ini melatih kemampuan mengelola hubungan dua arah antar-node dan menjaga konsistensi pointer saat struktur berubah. Konsep DLL berguna pada playlist, navigasi dua arah, undo-redo, dan struktur data yang membutuhkan akses ke elemen sebelum serta sesudahnya.

BAB II

PEMBAHASAN

Dasar Teori

Node DLL menyimpan data, pointer next, dan pointer prev. Head memiliki prev null, sementara tail memiliki next null. Traversal maju mengikuti next dan traversal mundur mengikuti prev.

Ketika node disisipkan atau dihapus, hubungan pada kedua sisi harus diperbarui. Kegagalan memperbarui salah satu pointer dapat memutus list atau menyebabkan arah traversal menghasilkan data yang tidak sesuai.

Program NodeDLL_2411532007 dan InsertDLL_2411532007

NodeDLL_2411532007 mendefinisikan data integer serta pointer next dan prev. InsertDLL_2411532007 menguji penambahan node pada awal, akhir, dan posisi tertentu.

Source Code NodeDLL_2411532007

```
1 package pekan6_2411532007;
2
3 public class NodeDLL_2411532007 {
4     int data_2007;
5     NodeDLL_2411532007 next_2007;
6     NodeDLL_2411532007 prev_2007;
7
8     public NodeDLL_2411532007 ( int data_2007) {
9         this.data_2007= data_2007;
10        this.next_2007 = null;
11        this.prev_2007 = null;
12    }
13 }
14
```

Source Code InsertDLL_2411532007

```

1 package pekan6_2411532007;
2
3 public class InsertDLL_2411532007 {
4     static NodeDLL_2411532007 insertBegin_2007(NodeDLL_2411532007 head_2007, int data_2007) {
5         NodeDLL_2411532007 new_node_2007 = new NodeDLL_2411532007(data_2007);
6         new_node_2007.next_2007 = head_2007;
7
8         if (head_2007 != null) {
9             head_2007.prev_2007 = new_node_2007;
10        }
11        return new_node_2007;
12    }
13
14    public static NodeDLL_2411532007 insertEnd_2007(NodeDLL_2411532007 head_2007, int newData_2007) {
15        NodeDLL_2411532007 newNode_2007 = new NodeDLL_2411532007(newData_2007);
16
17        if (head_2007 == null) {
18            head_2007 = newNode_2007;
19        }
20
21        else {
22            NodeDLL_2411532007 curr_2007 = head_2007;
23            while (curr_2007.next_2007 != null) {
24                curr_2007 = curr_2007.next_2007;
25            }
26            curr_2007.next_2007 = newNode_2007;
27            newNode_2007.prev_2007 = curr_2007;
28        }
29
30        return head_2007;
31    }
32
33    public static NodeDLL_2411532007 insertAtPosition_2007(NodeDLL_2411532007 head_2007, int pos_2007, int new_data_2007) {
34        NodeDLL_2411532007 new_node_2007 = new NodeDLL_2411532007(new_data_2007);
35        if (pos_2007 == 1) {
36            new_node_2007.next_2007 = head_2007;
37            if (head_2007 != null) {
38                head_2007.prev_2007 = new_node_2007;
39            }
40            head_2007 = new_node_2007;
41            return head_2007;
42        }
43        NodeDLL_2411532007 curr_2007 = head_2007;
44        NodeDLL_2411532007 curr_2007 = head_2007;
45
46        for (int i = 1; i < pos_2007 - 1 && curr_2007 != null; i++) {
47            curr_2007 = curr_2007.next_2007;
48        }
49
50        if (curr_2007 == null) {
51            System.out.println("Posisi Tidak Ada");
52            return head_2007;
53        }
54        new_node_2007.prev_2007 = curr_2007;
55        new_node_2007.next_2007 = curr_2007.next_2007;
56        curr_2007.next_2007 = new_node_2007;
57
58        if (new_node_2007.next_2007 != null) {
59            new_node_2007.next_2007.prev_2007 = new_node_2007;
60        }
61        return head_2007;
62    }
63
64    public static void printList_2007(NodeDLL_2411532007 head_2007) {
65        NodeDLL_2411532007 curr_2007 = head_2007;
66        while (curr_2007 != null) {
67            System.out.print(curr_2007.data_2007 + " <-> ");
68            curr_2007 = curr_2007.next_2007;
69        }
70        System.out.println();
71    }
72
73    public static void main (String args[]) {
74        NodeDLL_2411532007 head_2007 = new NodeDLL_2411532007(2);
75        head_2007.next_2007 = new NodeDLL_2411532007(3);
76        head_2007.next_2007.prev_2007 = head_2007;
77        head_2007.next_2007.next_2007 = new NodeDLL_2411532007(5);
78        head_2007.next_2007.next_2007.prev_2007 = head_2007.next_2007;
79
80        System.out.print("DLL Awal: ");
81        printList_2007(head_2007);
82        head_2007 = insertBegin_2007(head_2007, 1);
83        System.out.print("Simpul 1 ditambah di awal: ");
84
85        printList_2007(head_2007);
86
87        System.out.print("Simpul 6 ditambah di akhir: ");
88        int data_2007 = 6;
89        head_2007 = insertEnd_2007(head_2007, data_2007);
90        printList_2007(head_2007);
91
92        System.out.print("tambah node 4 di posisi 4: ");
93        int data2_2007 = 4;
94        int pos_2007 = 4;
95        head_2007 = insertAtPosition_2007(head_2007, pos_2007, data2_2007);
96        printList_2007(head_2007);
97    }
98 }

```

Penjelasan Operasi

1. insertBegin_2007() menghubungkan node baru sebelum head.

2. insertEnd_2007() menelusuri tail lalu menghubungkan node baru.
3. insertAtPosition_2007() menyisipkan node serta memperbarui next dan prev pada kedua sisi.
4. printList_2007() menelusuri list dari head ke tail.

Hasil Eksekusi

```
DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Analisis

DLL awal $2 \leftrightarrow 3 \leftrightarrow 5$ berubah menjadi $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 5$ setelah penambahan di awal. Nilai 6 ditambahkan di akhir dan nilai 4 disisipkan pada posisi keempat.

Hasil akhir $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5 \leftrightarrow 6$ menunjukkan bahwa pointer next dan prev berhasil diperbarui selama proses penyisipan.

Program PenelusuranDLL_2411532007

Program membentuk tiga node dan menghubungkannya dua arah, kemudian melakukan traversal maju dari head dan traversal mundur dari tail.

Source Code PenelusuranDLL_2411532007

```

1 package pekan6_2411532007;
2
3 public class PenelusuranDLL_2411532007 {
4     static void forwardTransversal_2007(NodeDLL_2411532007 head_2007) {
5         NodeDLL_2411532007 curr_2007 = head_2007;
6
7         while (curr_2007 != null) {
8             System.out.print(curr_2007.data_2007 + " <-> ");
9             curr_2007 = curr_2007.next_2007;
10        }
11
12        System.out.println();
13    }
14
15    static void backwardTransversal_2007(NodeDLL_2411532007 tail_2007) {
16        NodeDLL_2411532007 curr_2007 = tail_2007;
17
18        while (curr_2007 != null) {
19            System.out.print(curr_2007.data_2007 + " <-> ");
20            curr_2007 = curr_2007.prev_2007;
21        }
22        System.out.println();
23    }
24
25    public static void main (String[] args) {
26        NodeDLL_2411532007 head_2007 = new NodeDLL_2411532007(1);
27        NodeDLL_2411532007 second_2007 = new NodeDLL_2411532007(2);
28        NodeDLL_2411532007 third_2007 = new NodeDLL_2411532007(3);
29
30        head_2007.next_2007 = second_2007;
31        second_2007.prev_2007 = head_2007;
32        second_2007.next_2007 = third_2007;
33        third_2007.prev_2007 = second_2007;
34
35        System.out.println("Penelusuran Maju:");
36        forwardTransversal_2007(head_2007);
37
38        System.out.println("Penelusuran Mundur:");
39        backwardTransversal_2007(third_2007);
40    }
41 }
42 }

```

Hasil Eksekusi

```

<terminated> PenelusuranDLL_2411532007 [Java Application] C:\Users\ndaru\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin\javaw.exe (4 Jul
Penelusuran Maju:
1 <-> 2 <-> 3 <->
Penelusuran Mundur:
3 <-> 2 <-> 1 <->

```

Analisis

Traversal maju menghasilkan 1, 2, 3 dengan mengikuti next. Traversal mundur menghasilkan 3, 2, 1 dengan mengikuti prev.

Kedua hasil mengonfirmasi bahwa setiap pasangan node telah mempunyai hubungan timbal balik yang benar.

Program HapusDLL_2411532007

Program HapusDLL menguji penghapusan node head, tail, dan posisi tertentu pada DLL.

Source Code HapusDLL_2411532007

```

1 package pekan6_2411532007;
2
3 public class HapusDLL_2411532007 {
4     public static NodeDLL_2411532007 delHead_2007(NodeDLL_2411532007 head_2007) {
5         if ( head_2007 == null ) {
6             return null;
7         }
8
9         NodeDLL_2411532007 temp_2007 = head_2007;
10        head_2007 = head_2007.next_2007;
11
12        if ( head_2007 != null ) {
13            head_2007.prev_2007 = null;
14        }
15
16        return head_2007;
17    }
18
19    public static NodeDLL_2411532007 dellast_2007(NodeDLL_2411532007 head_2007) {
20        if ( head_2007 == null ) {
21            return null;
22        }
23        if ( head_2007.next_2007 == null ) {
24            return null;
25        }
26
27        NodeDLL_2411532007 curr_2007 = head_2007;
28        while ( curr_2007.next_2007 != null ) {
29            curr_2007 = curr_2007.next_2007;
30        }
31
32        if ( curr_2007.prev_2007 != null ) {
33            curr_2007.prev_2007.next_2007 = null;
34        }
35
36        return head_2007;
37    }
38
39    public static NodeDLL_2411532007 delPos_2007(NodeDLL_2411532007 head_2007, int pos_2007) {
40        if ( head_2007 == null ) {
41            return head_2007;
42        }
43        NodeDLL_2411532007 curr_2007 = head_2007;
44        NodeDLL_2411532007 curr_2007 = head_2007;
45
46        for ( int i = 1; curr_2007 != null && i < pos_2007; i++ ) {
47            curr_2007 = curr_2007.next_2007;
48        }
49
50        if ( curr_2007 == null ) {
51            return head_2007;
52        }
53
54        if ( curr_2007.prev_2007 != null ) {
55            curr_2007.prev_2007.next_2007 = curr_2007.next_2007;
56        }
57
58        if ( curr_2007.next_2007 != null ) {
59            curr_2007.next_2007.prev_2007 = curr_2007.prev_2007;
60        }
61
62        if ( head_2007 == curr_2007 ) {
63            head_2007 = curr_2007.next_2007;
64        }
65
66        return head_2007;
67    }
68
69    public static void printlist_2007(NodeDLL_2411532007 head_2007) {
70        NodeDLL_2411532007 curr_2007 = head_2007;
71        while ( curr_2007 != null ) {
72            System.out.print(curr_2007.data_2007 + " <-> ");
73            curr_2007 = curr_2007.next_2007;
74        }
75        System.out.println();
76    }
77
78    public static void main ( String [] args) {
79        NodeDLL_2411532007 head_2007 = new NodeDLL_2411532007(1);
80        head_2007.next_2007 = new NodeDLL_2411532007(2);
81        head_2007.next_2007.prev_2007 = head_2007;
82        head_2007.next_2007.next_2007 = new NodeDLL_2411532007(3);
83        head_2007.next_2007.next_2007.prev_2007 = head_2007.next_2007;
84        head_2007.next_2007.next_2007.next_2007 = new NodeDLL_2411532007(4);
85        head_2007.next_2007.next_2007.next_2007.prev_2007 = head_2007.next_2007.next_2007;
86        head_2007.next_2007.next_2007.next_2007.next_2007 = new NodeDLL_2411532007(5);

```

```

84     head_2007.next_2007.next_2007.next_2007.prev_2007 = head_2007.next_2007.next_2007;
85     head_2007.next_2007.next_2007.next_2007.next_2007 = new NodeDLL_2411532007(5);
86     head_2007.next_2007.next_2007.next_2007.next_2007.prev_2007 = head_2007.next_2007.next_2007;
87
88     System.out.print("DLL Awal: ");
89     printList_2007(head_2007);
90
91     System.out.print("Setelah head Dihapus: ");
92     head_2007 = delHead_2007(head_2007);
93     printList_2007(head_2007);
94
95     System.out.print("Setelah node terakhir Dihapus: ");
96     head_2007 = delLast_2007(head_2007);
97     printList_2007(head_2007);
98
99     System.out.print("Menghapus node ke 2: ");
100    head_2007 = delPos_2007(head_2007, 2);
101    printList_2007(head_2007);
102 }
103 }
104

```

Hasil Eksekusi

```

DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head Dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir Dihapus: 2 <-> 3 <-> 4 <->
Menghapus node ke 2: 2 <-> 4 <->

```

Analisis

List awal $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5$ berubah menjadi $2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5$ setelah head dihapus. Penghapusan tail menghilangkan 5, lalu penghapusan node kedua menghilangkan 3.

List akhir $2 \leftrightarrow 4$ menunjukkan bahwa next node 2 dan prev node 4 telah disambungkan kembali setelah node target dilepas.

BAB III

KESIMPULAN

Praktikum 6 berhasil menerapkan Double Linked List dengan traversal dua arah, penambahan, dan penghapusan node. Pointer next dan prev harus selalu diperbarui secara berpasangan.

DLL menyediakan fleksibilitas navigasi yang lebih tinggi dibanding SLL, meskipun membutuhkan ruang tambahan dan pengelolaan referensi yang lebih teliti. Playlist musik memperlihatkan kegunaan traversal maju dan mundur secara langsung.

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Sierra, K., Bates, B., & Gee, T. (2022). Head First Java (3rd ed.). O'Reilly Media.

Oracle. (2024). Java Platform, Standard Edition API Specification. Oracle Corporation.

LAPORAN TUGAS

Laporan tugas Praktikum 6 menerapkan Double Linked List pada playlist musik. Class lagu_2411532007 menjadi node yang menyimpan judul, penyanyi, next, dan prev. Class musik_2411532007 mengelola penambahan, penghapusan awal, traversal dua arah, pencarian, dan menu interaktif.

Tujuan Tugas

1. Membuat node lagu dengan pointer next dan prev.
2. Menambahkan lagu ke bagian akhir playlist.
3. Menghapus lagu pada awal playlist.
4. Menampilkan playlist maju dan mundur.
5. Mencari lagu berdasarkan judul.

Class lagu_2411532007

Class lagu_2411532007 merepresentasikan satu node playlist. Atribut judul dan penyanyi menyimpan data lagu, sedangkan next dan prev menghubungkan node dalam dua arah.

Source Code

```

1 package pekan6_2411532007;
2
3 public class lagu_2411532007 {
4     String judul_2007;
5     String penyanyi_2007;
6     lagu_2411532007 next_2007;
7     lagu_2411532007 prev_2007;
8
9     public lagu_2411532007(String j_2007, String p_2007) {
10         this.judul_2007 = j_2007;
11         this.penyanyi_2007 = p_2007;
12         this.next_2007 = null;
13         this.prev_2007 = null;
14     }
15
16     public void setJ_2007(String j_2007) {
17         this.judul_2007 = j_2007;
18     }
19
20     public void setP_2007(String p_2007) {
21         this.penyanyi_2007 = p_2007;
22     }
23
24     public String getJ_2007() {
25         return judul_2007;
26     }
27
28     public String getP_2007() {
29         return penyanyi_2007;
30     }
31 }
32

```

Penjelasan

1. judul_2007 dan penyanyi_2007 menyimpan informasi lagu.
2. next_2007 menunjuk lagu berikutnya dan prev_2007 menunjuk lagu sebelumnya.
3. Konstruktor menginisialisasi data dan menetapkan kedua pointer sebagai null.
4. Getter dan setter menyediakan akses terhadap judul dan penyanyi.

Class musik_2411532007

Class musik_2411532007 menjadi pengelola playlist sekaligus driver program. Head playlist disimpan pada variabel item_2007 dalam method main.

Source Code

```

1 package pekan6_2411532007;
2 import java.util.Scanner;
3
4 public class musik_2411532007 {
5
6     public static lagu_2411532007 tambahLagu_2007(lagu_2411532007 l_2007, String j_2007, String p_2007) {
7         lagu_2411532007 lg_2007 = new lagu_2411532007(j_2007, p_2007);
8
9         if (l_2007 == null) return lg_2007;
10
11         lagu_2411532007 curr_2007 = l_2007;
12         while (curr_2007.next_2007 != null) {
13             curr_2007 = curr_2007.next_2007;
14         }
15
16         curr_2007.next_2007 = lg_2007;
17         lg_2007.prev_2007 = curr_2007;
18         return l_2007;
19     }
20
21     public static lagu_2411532007 hapusLaguAwal_2007(lagu_2411532007 l_2007) {
22         if (l_2007 == null) {
23             return null;
24         }
25
26         lagu_2411532007 temp_2007 = l_2007;
27         l_2007 = l_2007.next_2007;
28
29         if (l_2007 != null) {
30             l_2007.prev_2007 = null;
31         }
32
33         temp_2007.next_2007 = null;
34         return l_2007;
35     }
36
37     public static void tampilMaju_2007(lagu_2411532007 l_2007) {
38         if (l_2007 == null) {
39             System.out.println("Playlist kosong.");
40             return;
41         }
42
43         lagu_2411532007 curr_2007 = l_2007;
44         while (curr_2007 != null) {
45             System.out.print(curr_2007.getj_2007() + " " + curr_2007.getp_2007() + " <-> ");
46             curr_2007 = curr_2007.next_2007;
47         }
48         System.out.println();
49     }
50
51     public static void tampilMundur_2007(lagu_2411532007 l_2007) {
52         if (l_2007 == null) {
53             System.out.println("Playlist kosong.");
54             return;
55         }
56
57         lagu_2411532007 curr_2007 = l_2007;
58         while (curr_2007.next_2007 != null) {
59             curr_2007 = curr_2007.next_2007;
60         }
61
62         while (curr_2007 != null) {
63             System.out.print(curr_2007.getj_2007() + " " + curr_2007.getp_2007() + " <-> ");
64             curr_2007 = curr_2007.prev_2007;
65         }
66         System.out.println();
67     }
68
69     public static boolean cariLagu_2007(lagu_2411532007 l_2007, String j_2007) {
70         lagu_2411532007 curr_2007 = l_2007;
71         while (curr_2007 != null) {
72             if (curr_2007.judul_2007.equalsIgnoreCase(j_2007)) {
73                 System.out.println("Judul Lagu: " + curr_2007.getj_2007());
74                 System.out.println("Penyanyi : " + curr_2007.getp_2007());
75                 return true;
76             }
77             curr_2007 = curr_2007.next_2007;
78         }
79         System.out.println("tidak ada lagu yang anda cari di playlist");
80         return false;
81     }
82
83     public static void main(String[] args) {
84         lagu_2411532007 item_2007 = null;
85         Scanner input_2007 = new Scanner(System.in);

```



```

85 Scanner input_2007 = new Scanner(System.in);
86 int pilihan_2007 = 0;
87
88 do {
89     System.out.println("\n=== playlist Musik Nim:2411532007 ===");
90     System.out.println("1. Tambah lagu");
91     System.out.println("2. Hapus lagu awal");
92     System.out.println("3. lihat playlist (maju)");
93     System.out.println("4. lihat playlist (mundur)");
94     System.out.println("5. Cari lagu");
95     System.out.println("6. Keluar");
96     System.out.print("Pilihan: ");
97
98     if (input_2007.hasNextInt()) {
99         pilihan_2007 = input_2007.nextInt();
100        input_2007.nextLine();
101    } else {
102        System.out.println("Input tidak valid. Masukkan angka.");
103        input_2007.nextLine();
104        continue;
105    }
106
107    switch (pilihan_2007) {
108        case 1:
109            System.out.print("Masukkan Judul lagu: ");
110            String j_2007 = input_2007.nextLine();
111            System.out.print("Masukkan Nama Penyanyi: ");
112            String p_2007 = input_2007.nextLine();
113
114            item_2007 = tambahLagu_2007(item_2007, j_2007, p_2007);
115            System.out.println("lagu berhasil ditambahkan");
116            break;
117
118        case 2:
119            item_2007 = hapusLaguAwal_2007(item_2007);
120            break;
121
122        case 3:
123            tampilMaju_2007(item_2007);
124            break;
125
126        case 4:
127            tampilMundur_2007(item_2007);
128            break;
129
130        case 5:
131            System.out.print("Masukkan judul lagu yang dicari: ");
132            String i_2007 = input_2007.nextLine();
133            cariLagu_2007(item_2007, i_2007);
134            break;
135
136        case 6:
137            System.out.println("Program selesai");
138            break;
139
140        default:
141            System.out.println("Pilihan tidak valid.");
142            break;
143    }
144
145    } while (pilihan_2007 != 6);
146
147    input_2007.close();
148 }
149 }

```

Penjelasan

1. tambahLagu_2007() menambahkan node pada tail dan mengatur hubungan prev.
2. hapusLaguAwal_2007() memindahkan head serta menetapkan prev head baru menjadi null.
3. tampilMaju_2007() mengikuti next dari head ke tail.
4. tampilMundur_2007() mencari tail lalu mengikuti prev menuju head.
5. cariLagu_2007() membandingkan judul tanpa membedakan kapital.

Hasil Eksekusi Laporan Tugas

```

=== playlist Musik Nim:2411532007 ===
1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar
Pilihan: 1
Masukkan Judul lagu: secukupnya
Masukkan Nama Penyanyi: hindia
lagu berhasil ditambahkan

=== playlist Musik Nim:2411532007 ===
1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar
Pilihan: 1
Masukkan Judul lagu: evaluasi
Masukkan Nama Penyanyi: hindia
lagu berhasil ditambahkan

=== playlist Musik Nim:2411532007 ===
1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar
Pilihan: 3
secukupnya hindia <-> evaluasi hindia <->

=== playlist Musik Nim:2411532007 ===
1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
=== playlist Musik Nim:2411532007 ===
1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar
Pilihan: 4
evaluasi hindia <-> secukupnya hindia <->

=== playlist Musik Nim:2411532007 ===
1. Tambah lagu
2. Hapus lagu awal
3. lihat playlist (maju)
4. lihat playlist (mundur)
5. Cari lagu
6. Keluar
Pilihan:

```

Analisis Hasil

Lagu Secukupnya dan Evaluasi ditambahkan secara berurutan. Traversal maju menampilkan Secukupnya lalu Evaluasi, sedangkan traversal mundur menghasilkan urutan sebaliknya. Hal ini menunjukkan hubungan next dan prev berfungsi.

Pencarian judul Evaluasi menemukan data penyanyi Hindia. Setelah lagu awal dihapus, playlist hanya menyisakan Evaluasi. Operasi tersebut memindahkan head dan memutus pointer prev pada head baru.

Kesimpulan Laporan Tugas

Class lagu_2411532007 dan musik_2411532007 berhasil menerapkan playlist berbasis DLL. Program mendukung penambahan, penghapusan awal, traversal maju-mundur, dan pencarian lagu.

Saran

Program dapat dikembangkan dengan penghapusan berdasarkan judul, penyisipan pada posisi tertentu, pemutaran next/previous, penyimpanan durasi, pengurutan playlist, dan pemisahan class driver agar tanggung jawab program lebih modular.